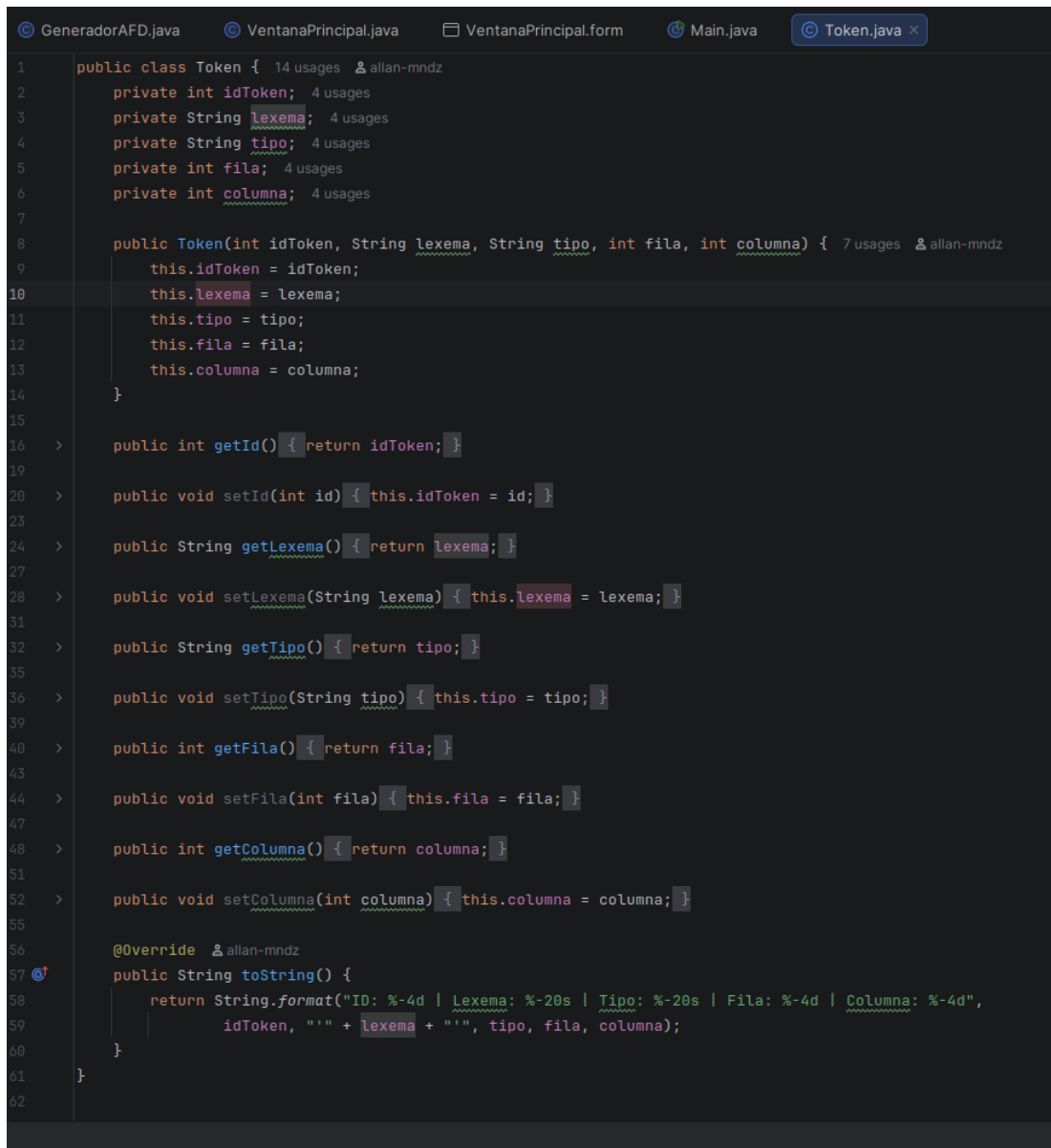


Manual Técnico

CLASE TOKEN

Primero tenemos los objetos para nuestro analizador léxico

Nuestro primer objeto es la del Token

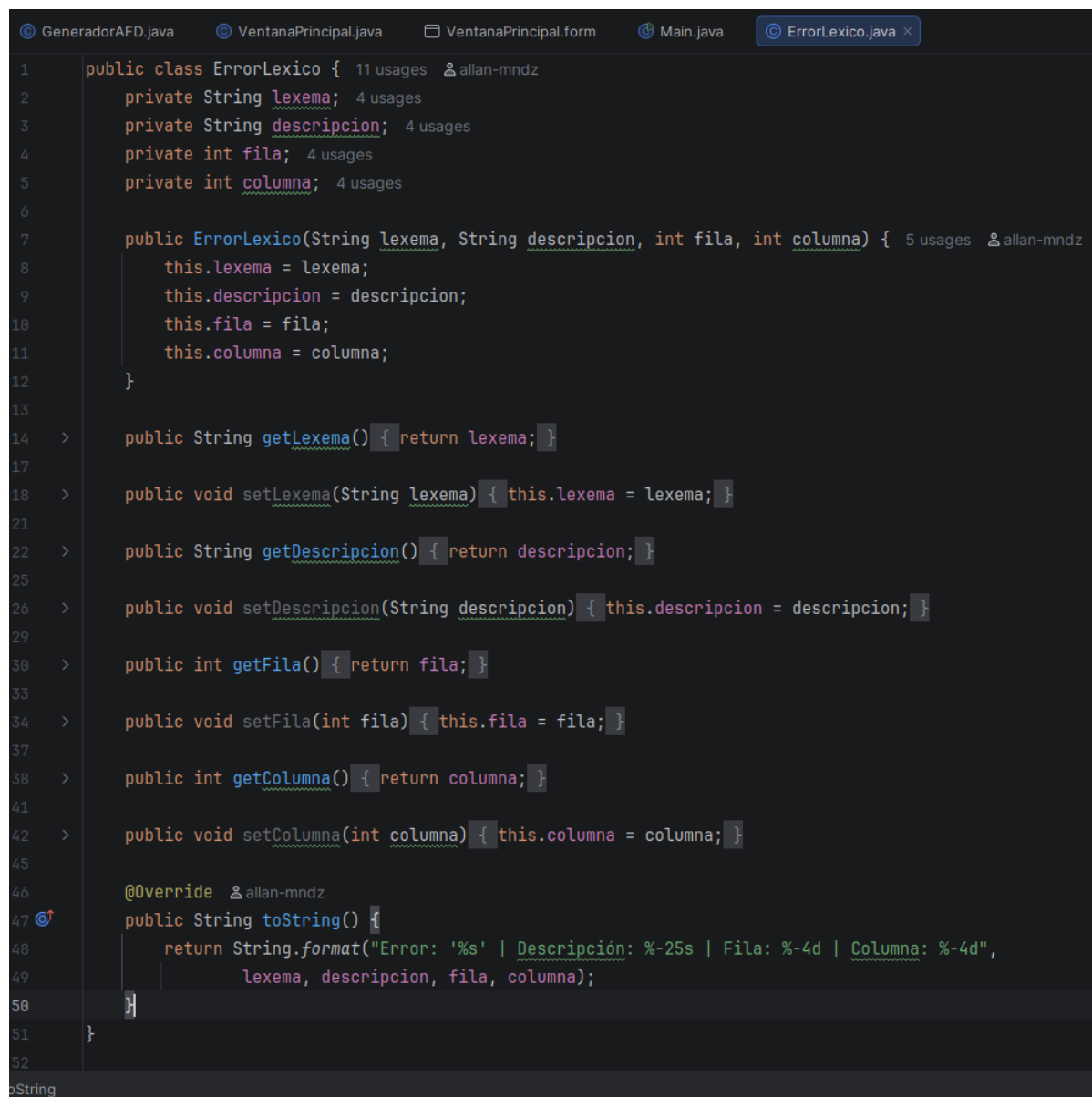


```
1 public class Token { 14 usages  allan-mndz
2     private int idToken; 4 usages
3     private String lexema; 4 usages
4     private String tipo; 4 usages
5     private int fila; 4 usages
6     private int columna; 4 usages
7
8     public Token(int idToken, String lexema, String tipo, int fila, int columna) { 7 usages  allan-mndz
9         this.idToken = idToken;
10        this.lexema = lexema;
11        this.tipo = tipo;
12        this.fila = fila;
13        this.columna = columna;
14    }
15
16    > public int getId() { return idToken; }
19
20    > public void setId(int id) { this.idToken = id; }
23
24    > public String getLexema() { return lexema; }
27
28    > public void setLexema(String lexema) { this.lexema = lexema; }
31
32    > public String getTipo() { return tipo; }
35
36    > public void setTipo(String tipo) { this.tipo = tipo; }
39
40    > public int getFila() { return fila; }
43
44    > public void setFila(int fila) { this.fila = fila; }
47
48    > public int getColumna() { return columna; }
51
52    > public void setColumna(int columna) { this.columna = columna; }
55
56    @Override  allan-mndz
57    @ public String toString() {
58        return String.format("ID: %-4d | Lexema: %-20s | Tipo: %-20s | Fila: %-4d | Columna: %-4d",
59            idToken, "" + lexema + "", tipo, fila, columna);
60    }
61 }
62
```

Tenemos los atributos necesarios, su constructor, metodos getters y setters, y un metodo toString nos sirve para imprimir los resultados ordenados finales del PromptZal.

CLASE ERROR LEXICO

De igual manera tenemos nuestro objeto de los errores léxicos.



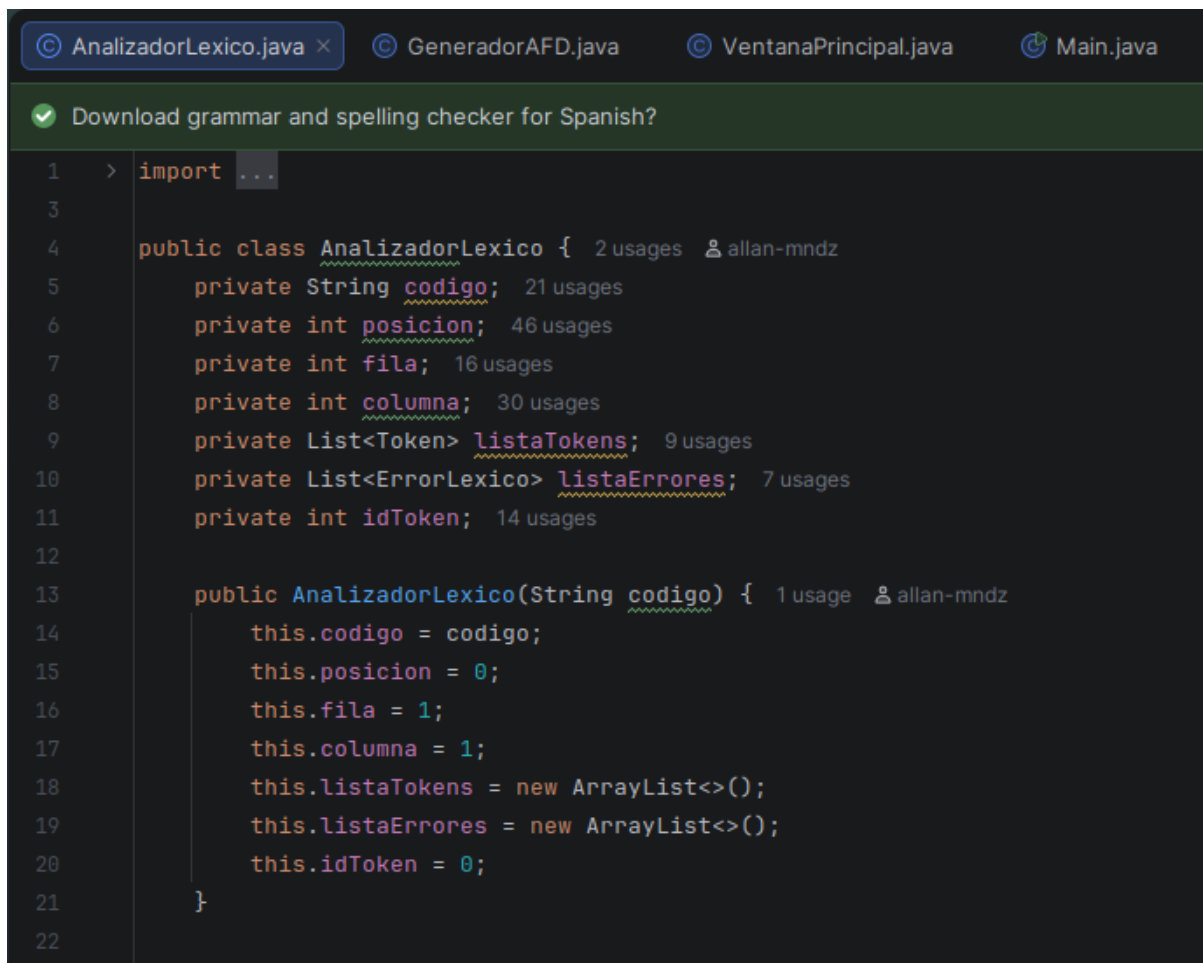
```
1 public class ErrorLexico { 11 usages  allan-mndz
2     private String lexema; 4 usages
3     private String descripcion; 4 usages
4     private int fila; 4 usages
5     private int columna; 4 usages
6
7     public ErrorLexico(String lexema, String descripcion, int fila, int columna) { 5 usages  allan-mndz
8         this.lexema = lexema;
9         this.descripcion = descripcion;
10        this.fila = fila;
11        this.columna = columna;
12    }
13
14    > public String getLexema() { return lexema; }
17
18    > public void setLexema(String lexema) { this.lexema = lexema; }
21
22    > public String getDescripcion() { return descripcion; }
25
26    > public void setDescripcion(String descripcion) { this.descripcion = descripcion; }
29
30    > public int getFila() { return fila; }
33
34    > public void setFila(int fila) { this.fila = fila; }
37
38    > public int getColumna() { return columna; }
41
42    > public void setColumna(int columna) { this.columna = columna; }
45
46    @Override  allan-mndz
47    public String toString() {
48        return String.format("Error: '%s' | Descripción: %-25s | Fila: %-4d | Columna: %-4d",
49            lexema, descripcion, fila, columna);
50    }
51 }
52
String
```

De misma manera que el anterior objeto tenemos sus atributos, constructor, métodos getters y setters, y el método toString para imprimir los resultados finales ordenados.

CLASE ANALIZADOR LEXICO

Tenemos la clase “Principal” de todo el programa que es el del Analizador Lexico

Se compone por los siguientes atributos con su constructor



The screenshot shows an IDE with four tabs: AnalizadorLexico.java, GeneradorAFD.java, VentanaPrincipal.java, and Main.java. A green notification bar at the top says "Download grammar and spelling checker for Spanish?". The code in the AnalizadorLexico.java tab is as follows:

```
1  > import ...
3
4  public class AnalizadorLexico { 2 usages  allan-mndz
5      private String codigo; 21 usages
6      private int posicion; 46 usages
7      private int fila; 16 usages
8      private int columna; 30 usages
9      private List<Token> listaTokens; 9 usages
10     private List<ErrorLexico> listaErrores; 7 usages
11     private int idToken; 14 usages
12
13     public AnalizadorLexico(String codigo) { 1 usage  allan-mndz
14         this.codigo = codigo;
15         this.posicion = 0;
16         this.fila = 1;
17         this.columna = 1;
18         this.listaTokens = new ArrayList<>();
19         this.listaErrores = new ArrayList<>();
20         this.idToken = 0;
21     }
22
```

Método Analizar

Tenemos el primer método que es el de analizar, este método nos sirve para identificar cuando hay un salto de línea, espacios entre palabras, el tipo de palabras que estaremos analizando, si tiene cierta o cumple cierta condición entonces ira a cierto método para analizar esa palabra o expresión completa.

```

73 public void analizar() { // usage & alan-mndz
74     int longitud = codigo.length(); // nos dice cuantas letras tiene el archivo en total
75
76     while(posicion < longitud){
77         // el char almacena un unico caracter
78         char c = codigo.charAt(posicion); // saca la letra actual y la guarda en la variable
79
80         if(c == '\n'){
81             fila++;
82             columna = 1;
83             posicion++;
84             continue;
85         } else if(c == '\r'){
86             posicion++;
87             continue;
88         } else if(c == '\t' || c == ' '){
89             columna++;
90             posicion++;
91             continue;
92         }
93
94         if(Character.isLetter(c) || c == '_'){
95             extraerPalabra();
96         } else if (c == '@') {
97             extraerDirectiva();
98         } else if (c == '"') { // para las cadenas de texto
99             extraerCadena();
100        } else if (Character.isDigit(c)) {
101            extraerNumero();
102        } else if (c == '[' || c == ']' || c == '(' || c == ')' || c == ',' || c == '=' || c == '+' || c == '-'){ //si la letra es uno de estos simbolos que entre
103            extraerSimboloUnico(c);
104        } else if (c == '-') { // sirve para guiar al programa si es un error o es la primera mitad de la flecha ->
105            extraerFlecha0Guion();
106        } else if (c == '/') { // sirve para guiar al programa si es un comentario, un multilinea o un error
107            extraerComentario0Division();
108        } else {
109            listaErrores.add(new ErrorLexico(String.valueOf(c), descripcion: "Carácter no reconocido", fila, columna));
110            posicion++;
111            columna++;
112        }
113    }
114 }

```

¿cómo sabemos hasta donde leer? Esto es gracias a

Int longitud = codigo.length(); // nos dice cuántas letras tiene el archivo en total

Método extraer Palabra

El primer método es para cuando tenemos un identificador, estas pueden ser una Palabra Reservada, Comando de IA y un Conector.

Primero tenemos que tomar en cuenta donde inicia la palabra esto lo hacemos con

Int ColumnaInicio = columna;

Esto para recordar en que columna inicia la palabra o el -

¿Y Fila? Las filas solo nos sirven para cuando haya un salto de línea (enter).

Luego tenemos que ir guardando letra por letra en una caja de texto, ¿cómo lo hacemos? Esto se hace gracias a

String lexema = "";

```
66     private void extraerPalabra() { 1 usage 2 allan-mndz
67         int columnaInicio = columna; // sirve para recordar en que columna inicia las palabras o el -
68         String lexema = "";           // variable de texto vacio para ir guardando letra por letra
69         int longitud = codigo.length();
70     }
```

La condición

While (posicion < longitud)

Nos sirve para no sobrepasar las letras en total que trae el archivo .pz y leer únicamente las letras que trae.

Char actual = codigo.charAt(posicion)

Nos sirve para ir leyendo letra por letra e ir guardando es letra en la variable posición.

```
71         while(posicion < longitud){
72             char actual = codigo.charAt(posicion);
73
74             //ve si es una letra del abecedario o un numero
75             if (Character.isLetterOrDigit(actual) || actual == '_') {
76                 lexema = lexema + actual;
77                 posicion++;
78                 columna++;
79             }else {
80                 break;
81             }
82         }
83     }
```

Después de analizar la palabra dependiendo que palabra sea sera el tipo de token que se nos guardara

```

83
84     String tipoToken = "Identificador";
85
86     if (lexema.equals("AGENTE") || lexema.equals("contexto") || lexema.equals("variable") || lexema.equals("EJECUTAR") || lexema.equals("EXPORTAR") || lexema.equals("CODIFICAR")) {
87
88         tipoToken = "Palabra reservada";
89
90     } else if (lexema.equals("PREGUNTAR") || lexema.equals("GENERAR") || lexema.equals("RESUMIR") || lexema.equals("ANALIZAR") || lexema.equals("TRADUCIR") ||
91         lexema.equals("CLASIFICAR") || lexema.equals("EXTRAER") || lexema.equals("CARGAR")) {
92
93         tipoToken = "Comando de IA";
94
95     } else if (lexema.equals("SOBRE") || lexema.equals("DESDE") || lexema.equals("EN") || lexema.equals("COMO")) {
96
97         tipoToken = "Conector";
98
99     }

```

Y por último lo guardamos en la lista de tokens

```

100
101     listaTokens.add(new Token(idToken, lexema, tipoToken, fila, columnaInicio)); // guardamos el token en la lista "oficial"
102     idToken++;
103 }

```

Método extraer Directiva

Este método es similar al anterior usa lo mismo para poder identificar las palabras con la diferencia que estas palabras siempre inician con una “@” y que todos los caracteres son letras.

```

105     private void extraerDirectiva(){ 1 usage @ allan-mndz
106         int columnaInicio = columna; // sirve para recordar en que columna empeco la palabra con el @
107         String lexema = "";
108         int longitud = codigo.length();
109
110         lexema = lexema + codigo.charAt(posicion); //va armando la palabra letra por letra
111         posicion++;
112         columna++;
113
114         //mientras haya letras se sigue avanzando
115         while(posicion < longitud && Character.isLetter(codigo.charAt(posicion))){
116             lexema = lexema + codigo.charAt(posicion);
117             posicion++;
118             columna++;
119         }
120
121         if (lexema.equals("@modelo") || lexema.equals("@rol") || lexema.equals("@formato")){
122             listaTokens.add(new Token(idToken, lexema, tipo: "Directiva", fila, columnaInicio));
123             idToken++;
124         }else{
125             listaErrores.add(new ErrorLexico(lexema, descripcion: "Directiva no reconocida", fila, columnaInicio));
126         }
127     }
128 }

```

Método extraer Cadena

Aquí los mismo con la diferencia que necesitamos una variable boolean para verifica si la cadena esta se cerró o solo tiene una parte de las comillas.

```
128
129     private void extraerCadena(){ 1 usage  & allan-mndz *
130         int columnaInicio = columna;
131         String lexema = "";
132         boolean cerrada = false;
133
134         posicion++;
135         columna++;
136
```

En el ciclo leemos todo hasta encontrar el cierre de las comillas, si no hay ningún cierre esto se guardar en la lista de errores caso contrario en la lista de tokens.

```
129     private void extraerCadena(){ 1 usage  & allan-mndz *
130         int columnaInicio = columna;
131         String lexema = "";
132         boolean cerrada = false;
133
134         posicion++;
135         columna++;
136
137         //leemos todo hasta encontrar el cierre comilla
138         while (posicion < codigo.length()){
139             //extraemos la letra actual para usarla en diferentes cuestiones
140             char actual = codigo.charAt(posicion);
141
142             if(actual == '"'){
143                 cerrada = true;
144                 posicion++;
145                 columna++;
146                 break;
147             }else if(actual == '\n'){
148                 break;
149             }else{
150                 lexema = lexema + actual;
151                 posicion++;
152                 columna++;
153             }
154         }
155
156         if (cerrada) {
157             listaTokens.add(new Token(idToken, lexema, tipo: "Literal de Cadena", fila, columnaInicio));
158             idToken++;
159         }else {
160             listaErrores.add(new ErrorLexico(lexema, descripcion: "Cadena sin cerrar", fila, columnaInicio));
161         }
162     }
```

Método extraer Numero

Aquí usamos la condición `Character.isDigit(actual)` para saber cuándo es un número y usamos la variable boolean para saber cuándo es un numero entero y cuando es un numero decimal

```
164 private void extraerNumero(){ 1 usage 3 allan-mndz *
165     int columnaInicio = columna;
166     String lexema = "";
167     int longitud = codigo.length();
168     boolean tienePunto = false;
169
170     while(posicion < longitud){
171         char actual = codigo.charAt(posicion);
172
173         if(Character.isDigit(actual)){ //si la letra actual es un dígito del 0-9 la agregamos y seguimos avanzando
174             lexema = lexema + actual;
175             posicion++;
176             columna++;
177         } else if (actual == '.' && !tienePunto) { // si tiene punto el tienePunto se activa
178             lexema = lexema + actual;
179             tienePunto = true;
180             posicion++;
181             columna++;
182         } else {
183             break;
184         }
185     }
186
187     if(tienePunto){
188         listaTokens.add(new Token(idToken, lexema, tipo: "Literal Decimal", fila, columnaInicio));
189     } else {
190         listaTokens.add(new Token(idToken, lexema, tipo: "Literal Entero", fila, columnaInicio));
191     }
192     idToken++;
193 }
```

Método extraer Símbolo Único

En este método primero necesitamos saber el símbolo exacto, con esto tomamos ese símbolo y lo introducimos al texto

Si el símbolo es un “=” o un “+” es un tipo Operador

Si el símbolo es de los otros símbolos válidos es de un tipo Delimitador.

```

195     private void extraerSimboloUnico(char c){ //recibe el simbolo exacto que mando el metodo principal
196         String lexema = String.valueOf(c); //toma el simbolo suelto y lo introduce al texto
197         String tipo;
198         if (c == '=' || c == '+'){
199             tipo = "Operador";
200         }else{
201             tipo = "Delimitador";
202         }
203
204         listaTokens.add(new Token(idToken, lexema, tipo, fila, columna));
205         idToken++;
206         posicion++;
207         columna++;
208     }

```

Método extraer Flecha Guion

Aquí necesitamos saber si el símbolo es una resta o una flecha, para saber cómo identificarlo nos adelantamos la posición esto con

posicion + 1

Si posicion + 1 == '>'

Es una flecha y como hicimos lo de posicion + 1 (nos movimos dos veces) entonces debemos indicarlos en los contadores por eso se pone

posicion += 2;

columna += 2;

```

210     private void extraerFlechaOGuion(){ 1 usage 2 allan-mndz
211         int longitud = codigo.length();
212         //posicion + 1 sirve para ver que hay en el siguiente espacio
213         if (posicion + 1 < longitud && codigo.charAt(posicion + 1) == '>') {
214
215             listaTokens.add(new Token(idToken, lexema: ">", tipo: "Conector", fila, columna));
216             idToken++;
217             //como leimos dos simbolos sumamos dos pasos hacia delante
218             posicion += 2;
219             columna += 2;
220         }else {
221             listaErrores.add(new ErrorLexico(lexema: ">", descripcion: "Carácter no reconocido", fila, columna));
222             posicion++;
223             columna++;
224         }
225     }
226

```

Método extraer Comentario o División

Este método nos sirve para ver si es un bloque de comentario (/* */) o un error.

Primero verificamos si es un comentario normal o un error.

Si es un comentario normal entra a este ciclo, no guardamos token porque las instrucciones se nos pidan que los comentarios se ignoren.

```
226
227     private void extraerComentarioODivision(){ 1 usage  @allan-mndz *
228         int longitud = codigo.length();
229
230         if (posicion + 1 < longitud && codigo.charAt(posicion + 1) == '/') { //verifica si es un comentario o un error
231             while (posicion < longitud && codigo.charAt(posicion) != '\n') {
232                 posicion++;
233                 columna++;
234             }
```

Si no es un comentario no mal sino es un comentario de bloque vamos a usar tres condiciones para poder cumplir esta parte.

Primero la condición while (posicion < longitud - 1) esto nos sirve para ver si al final de la longitud un espacio antes de terminar veamos que símbolo tiene es decir sirve para ver si el símbolo anterior es “*”

Necesitamos usar dos char, uno para lo que siempre lo hemos estado usando para ir guardando letra por letra y otro para saber cuál es el siguiente símbolo cuando nos topemos un “*” para ver si finaliza el bloque de comentario o no.

Si dentro de un bloque de comentario hay un salto de línea solamente vamos sumando la fila, posición y la columna a la posición 1 para que inicie a leer.

```

235     } else if (posicion + 1 < longitud && codigo.charAt(posicion + 1) == '*') {
236         posicion += 2;
237         columna += 2;
238         boolean cerrado = false; //sirve para encontrar el final del comentario
239
240         while (posicion < longitud - 1) { //sirve para buscar el */ para poder salir
241             char actual = codigo.charAt(posicion);
242             char siguiente = codigo.charAt(posicion + 1);
243
244             if (actual == '\n') {
245                 fila++;
246                 columna = 1;
247                 posicion++;

```

Si nos topamos con un “*” y el siguiente es un “/” entonces finaliza el bloque de comentario y la variable booleana pasa como verdadera y el break para salir del ciclo de forma inmediata.

```

248     } else if (actual == '*' && siguiente == '/') {
249         posicion += 2;
250         columna += 2;
251         cerrado = true;
252         break;

```

En dado caso que no haya salto de línea y no haya cierre del bloque de comentario solamente vamos sumando la posición y la columna para seguir leyendo el archivo.

```

253     } else {
254         posicion++;
255         columna++;
256     }
257 }

```

CLASE VENTANA PRINCIPAL

En la ventana principal extiende de JFrame porque es una ventana de swing para tener una vista más cómoda para trabajar.

Esta ventana está compuesta por los siguientes elementos

```
© GeneradorAFD.java  © VentanaPrincipal.java x  Main.java

1  import javax.swing.*;
2  import java.awt.event.ActionEvent;
3  import java.awt.event.ActionListener;
4  import java.io.File;
5  import java.io.FileWriter;
6  import java.io.IOException;
7  import java.util.ArrayList;
8  import java.util.HashMap;
9  import java.util.List;
10 import java.util.Map;
11
12 public class VentanaPrincipal extends JFrame {
13     private JButton btnAbrir;
14     private JButton btnGuardar;
15     private JButton btnAnalizar;
16     private JTextArea txtEntrada;
17     private JTextArea txtConsolaTokens;
18     private JTextArea txtConsolaErrores;
19     private JPanel panelPrincipal;
20     private JTabbedPane tabbedPane2;
21     private JButton btnReportes;
22     private JButton btnGenerarAFD;
23 }
```

También instanciamos estas variables que vamos a usar dentro de la clase

```
24     private List<Token> listaTokensActual = new ArrayList<>();
25     private List<ErrorLexico> listaErroresActual = new ArrayList<>();
26     private String codigoFuenteActual = "";
27 }
```

Botón Analizar

Primero verificamos que en la caja de texto no este vacío y luego llamamos a nuestra clase Analizador Léxico y nuestro método analizar

```

btnAnalizar.addActionListener(new ActionListener() { @Override no usages @allan-mndz *
@Override no usages @allan-mndz *
public void actionPerformed(ActionEvent e) {
    lexema = txtEntrada.getText();

    if (lexema.trim().isEmpty()) { // verifica si no hay texto o solo tiene espacios
        JOptionPane.showMessageDialog( parentComponent: null, message: "El editor está vacío.", title: "Advertencia", JOptionPane.WARNING_MESSAGE);
        return;
    }

    AnalizadorLexico analizador = new AnalizadorLexico(lexema);
    analizador.analizar();
}
}

```

Despues de eso hacemos un ciclo for para cada token y error con la condición de que por cada token/error que exista dentro de la lista tokens/errores se haga lo siguiente que sería poner los atributos de cada objeto.

```

52     StringBuilder reporteTokens = new StringBuilder();
53     reporteTokens.append("--- TOKENS RECONOCIDOS ---\n\n");
54
55     for (Token t : listaTokensActual) { // por cada token que exista dentro de mi listaTokens, haz lo siguiente:
56         reporteTokens.append("Lexema: ").append(t.getLexema()).append("\n")
57             .append(" | Tipo: ").append(t.getTipo())
58             .append(" | Fila: ").append(t.getFila())
59             .append(" | Columna: ").append(t.getColumna()).append("\n");
60     }
61     txtConsoleTokens.setText(reporteTokens.toString());
62
63     StringBuilder reporteErrores = new StringBuilder();
64     if (listaErroresActual.isEmpty()) {
65         reporteErrores.append("No se encontraron errores lexicos.\n");
66     } else {
67         reporteErrores.append("--- ERRORES ENCONTRADOS ---\n\n"); // por cada error que exista dentro de mi listaErrores, haz lo siguiente:
68         for (ErrorLexico err : listaErroresActual) {
69             reporteErrores.append("Error: ").append(err.getLexema()).append("\n")
70                 .append(" | Descripción: ").append(err.getDescripcion())
71                 .append(" | Fila: ").append(err.getFila())
72                 .append(" | Columna: ").append(err.getColumna()).append("\n");
73         }
74     }
75     txtConsoleErrores.setText(reporteErrores.toString());
76
77     JOptionPane.showMessageDialog( parentComponent: null, message: "Análisis completado.", title: "Listo", JOptionPane.INFORMATION_MESSAGE);
78 }
79 });

```

Botón Guardar

Con este botón podemos guardar el archivo que estemos modificando o creando dentro de la caja de texto en donde se analiza el archivo .pz todo esto es gracias al uso de JFileChooser y con una condición que el archivo debe tener la ruta .pz

```

82     btnGuardar.addActionListener(new ActionListener() { @allan-mndz
83         @Override no usages @allan-mndz
84         public void actionPerformed(ActionEvent actionEvent) {
85             JFileChooser guardar = new JFileChooser();
86             int archivoGuardado = guardar.showSaveDialog( parent: null);
87
88             if (archivoGuardado == JFileChooser.APPROVE_OPTION) {
89                 File archivo = guardar.getSelectedFile();
90
91                 String ruta = archivo.getPath();
92                 if(!ruta.endsWith(".pz")){
93                     archivo = new File( pathname: ruta + ".pz");
94                 }
95
96                 try(FileWriter fw = new FileWriter(archivo)){
97                     fw.write(txtEntrada.getText());
98                     JOptionPane.showMessageDialog( parentComponent: null, message: "Archivo .pz guardado con éxito");
99                 }catch (IOException e){
100                     JOptionPane.showMessageDialog( parentComponent: null, message: "Error al guardar el archivo");
101                 }
102             }
103         }
104     });
105
106
107

```

Botón Abrir

Este botón nos sirve para abrir explorador de archivos de la computadora y buscar un archivo para poder analizar y detectar sus tokens y sus errores, esto gracias también al JFileChooser

```

109     btnAbrir.addActionListener(new ActionListener() { @allan-mndz *
110         @Override no usages @allan-mndz *
111         public void actionPerformed(ActionEvent actionEvent) {
112             JFileChooser explorador = new JFileChooser();
113             int resultado = explorador.showOpenDialog( parent: null);
114
115             if (resultado == JFileChooser.APPROVE_OPTION) {
116                 File archivoSeleccionado = explorador.getSelectedFile();
117                 try {
118                     String contenido = Files.readString(archivoSeleccionado.toPath());
119                     txtEntrada.setText(contenido);
120                 }catch (Exception ex){
121                     JOptionPane.showMessageDialog( parentComponent: null, ex.getMessage());
122                 }
123             }
124         }
125     });

```

Botón Reportes

Primero se tiene que analizar un documento .pz para poder generar los reportes, después de analizarlo, nuevamente tenemos que elegir donde queremos guardar los reportes

```
127 btnReportes.addActionListener(new ActionListener() { @allan-mndz *
128 @Override no usages @allan-mndz *
129 public void actionPerformed(ActionEvent actionEvent) {
130     if (lexema.isEmpty() && listaTokensActual.isEmpty() && listaErroresActual.isEmpty()) {
131         JOptionPane.showMessageDialog(parentComponent: null, message: "Primero debes Analizar un documento .pz para poder generar reportes", title: "Advertencia", JOptionPane.INFORMATION_MESSAGE);
132         return;
133     }
134
135     // Abrimos el explorador para elegir la carpeta destino
136     JFileChooser explorador = new JFileChooser();
137     explorador.setDialogTitle("Selecciona la carpeta para guardar los reportes");
138     explorador.setFileSelectionMode(JFileChooser.DIRECTORIES_ONLY);
139
140     int seleccion = explorador.showSaveDialog(parent: null);
141
142     if (seleccion == JFileChooser.APPROVE_OPTION) {
143         // Obtenemos la ruta de la carpeta elegida
144         String rutaCarpeta = explorador.getSelectedFile().getAbsolutePath();
145
146         // Llamamos al metodo pasandole la nueva ruta
147         generarReportesHTML(listaTokensActual, listaErroresActual, lexema, rutaCarpeta);
148     }
149 }
150 });
```

Botón GenerarAFD

Con este boton podemos generar la imagen del Autómata Finito Deeterminista esto gracias a la clase GeneradorAFD que en un momento explicaremos cómo funciona, y podemos elegir donde guarda la imagen y el archivo .dot

```
151
152 btnGenerarAFD.addActionListener(new ActionListener() { @allan-mndz
153 @Override no usages @allan-mndz
154 public void actionPerformed(ActionEvent actionEvent) {
155     JFileChooser explorador = new JFileChooser();
156     explorador.setDialogTitle("Selecciona dónde guardar la imagen del autómata");
157     explorador.setFileSelectionMode(JFileChooser.DIRECTORIES_ONLY);
158
159     int seleccion = explorador.showSaveDialog(parent: null);
160
161     if (seleccion == JFileChooser.APPROVE_OPTION) {
162         String rutaCarpeta = explorador.getSelectedFile().getAbsolutePath();
163
164         GeneradorAFD generador = new GeneradorAFD();
165         generador.crearAFD(rutaCarpeta);
166     }
167 }
168 });
169 }
```

CLASE GENERADOR AFD

Aqui solamente necesitamos un método que es llamado crearAFD
String codigoDOT = "" sirve para almacenar la estructura completa del grafo

Shape se utiliza o significa “forma”. Graphviz lo utiliza para decidir que geometria utilizar.

Label se utiliza o significa “etiqueta”. Es lo que va a ir encima de la flecha que estara apuntando a otro estado

```

5 public class GeneradorAFD { 2 usages  &allan-mndz *
6
7     public void crearAFD(String rutaCarpeta) { 1 usage  &allan-mndz *
8
9         // almacena la estructura completa del grafo y utiliza bloques de texto (""") multilinea
10        String codigoDOT = ""
11
12        //inicializa el grafo dirigido con flechas
13        digraph AFD {
14
15            // flecha del estado inicial
16            inicio [shape = point];
17
18            // nodo del estado inicial
19            S0 [shape = circle];
20
21            // nodos del estado final
22            S1 [shape = doublecircle];
23            S2 [shape = doublecircle];
24            S4 [shape = doublecircle];
25            S6 [shape = doublecircle];
26            S8 [shape = doublecircle];
27            S11 [shape = doublecircle];
28            S14 [shape = doublecircle];
29            S16 [shape = doublecircle];
30            S17 [shape = doublecircle];
31            S18 [shape = doublecircle];
32            S19 [shape = doublecircle];
33            S20 [shape = doublecircle];
34            S21 [shape = doublecircle];
35            S22 [shape = doublecircle];
36            S23 [shape = doublecircle];
37            S24 [shape = doublecircle];
38

```

```
39 // nodos normales
40 S3 [shape = circle];
41 S5 [shape = circle];
42 S7 [shape = circle];
43 S9 [shape = circle];
44 S10 [shape = circle];
45 S12 [shape = circle];
46 S13 [shape = circle];
47 S15 [shape = circle];
48
49 // flecha de entrada
50 inicio -> S0 [label = " inicio "];
51
52
53 // identificadores, palabras reservadas y comandos
54
55 S0 -> S1 [label = " Letra "];
56 S1 -> S1 [label = " Letra, Digito, _ "];
57
58 // Numeros enteros y decimales
59
60 S0 -> S2 [label = " Digito "];
61 S2 -> S2 [label = " Digito "];
62 S2 -> S3 [label = " . "];
63 S3 -> S4 [label = " Digito "];
64 S4 -> S4 [label = " Digito "];
65
66 // Directivas
67
68 S0 -> S5 [label = " @ "];
69 S5 -> S6 [label = " Letra "];
70 S6 -> S6 [label = " Letra "];
71
```

```

72 // Cadenas de texto
73
74 S0 -> S7 [label = " Comilla doble "];
75 S7 -> S7 [label = " Cualquier cosa "];
76 S7 -> S8 [label = " Comilla doble "];
77
78 // Comentarios
79
80 S0 -> S9 [label = " / "];
81 S9 -> S10 [label = " / "];
82 S10 -> S10 [label = " Cualquier cosa "];
83 S10 -> S11 [label = " Salto de línea (enter) "];
84
85 // comentario de bloque
86
87 S9 -> S12 [label = " * "];
88 S12 -> S12 [label = " Cualquier cosa "];
89 S12 -> S13 [label = " * "];
90 S13 -> S12 [label = " Cualquier cosa "];
91 S13 -> S14 [label = " / "];
92
93
94 // conector con flecha
95
96 S0 -> S15 [label = " - "];
97 S15 -> S16 [label = " > "];
98
99 // operadores
100
101 S0 -> S17 [label = " = "];
102 S0 -> S18 [label = " + "];
103
104 // delimitadores
105
106 S0 -> S19 [label = " { "];
107 S0 -> S20 [label = " } "];
108 S0 -> S21 [label = " ( "];
109 S0 -> S22 [label = " ) "];
110 S0 -> S23 [label = " , "];
111 S0 -> S24 [label = " : "];
112 }
113 """,

```

Después utilizamos un try-catch por cualquier error.

```

114     try {
115         // sirve para guardar la imagen y el dot directamente con sus nombres donde sea que lo guardemos
116         String rutaDot = rutaCarpeta + File.separator + "automata.dot";
117         String rutaPng = rutaCarpeta + File.separator + "automata.png";
118
119         //Creamos el archivo .dot en la carpeta elegida
120         File archivoDot = new File(rutaDot);
121         FileWriter escritor = new FileWriter(archivoDot);
122         escritor.write(codigoDOT);
123         escritor.close(); // guarda los cambios
124
125         // Le pasamos las nuevas rutas exactas a Graphviz
126         ProcessBuilder pb = new ProcessBuilder("dot", "-Tpng", rutaDot, "-o", rutaPng); // comunicacion entre java y la terminal es como escribir eso en la terminal
127         pb.inheritIO(); // la salida de lo que diria la terminal nos lo dice en nuestra consola
128         Process proceso = pb.start();
129         proceso.waitFor(); // sirve para pausar la ejecucion y asi darle tiempo a Graphviz para terminar el dibujo
130
131         JOptionPane.showMessageDialog(parentComponent: null, message: "Grafo generado exitosamente en:\n" + rutaPng, title: "Graphviz", JOptionPane.INFORMATION_MESSAGE);
132
133     } catch (Exception e) {
134         JOptionPane.showMessageDialog(parentComponent: null, message: "Error al generar la imagen del AFD" + e.getMessage(), title: "Error", JOptionPane.ERROR_MESSAGE);
135     }
136 }
137 }
138

```

AFD

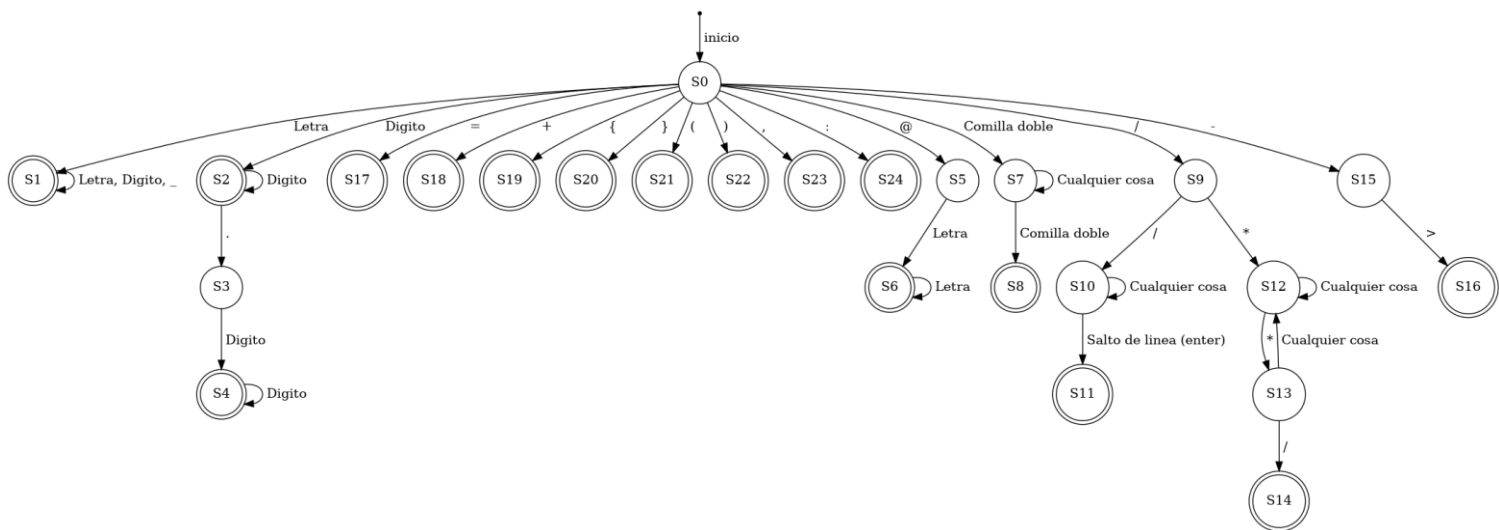


Tabla de Tokens

token	Gramática regular	Estado final	Condición de aceptación
Identificadores y palabras reservadas	$S0 \rightarrow LS1$ $S1 \rightarrow LS1$ $S1 \rightarrow DS1$ $S1 \rightarrow -S1$ $S1 \rightarrow \epsilon$	s1	Al leer cualquier carácter que contenga letra, dígito o guión bajo
Numeros Enteros	$S0 \rightarrow DS2$ $S2 \rightarrow DS2$ $S2 \rightarrow \epsilon$	S2	Al encontrar un espacio, símbolo o carácter no numérico (distinto de un punto).
Números Decimales	$S2 \rightarrow .S3$ $S3 \rightarrow DS4$ $S4 \rightarrow DS4$ $S4 \rightarrow \epsilon$	S4	Al terminar de leer la parte decimal y encontrar un espacio o delimitador.
Directivas	$S0 \rightarrow @S5$ $S5 \rightarrow LS6$ $S6 \rightarrow LS6$ $S6 \rightarrow \epsilon$	S6	Al leer un carácter distinto a una letra tras el arroba.
Cadena de Texto	$S0 \rightarrow "S7$ $S7 \rightarrow CS7$ $S7 \rightarrow "S8$ $S8 \rightarrow \epsilon$	S8	Al chocar con el símbolo de comillas dobles de cierre (").
Comentario de Línea	$S0 \rightarrow /S9$ $S9 \rightarrow /S10$ $S10 \rightarrow CS10$ $S10 \rightarrow \backslash nS11$ $S11 \rightarrow \epsilon$	S11	Al detectar un salto de línea (Enter).
Comentarios de Bloque	$S9 \rightarrow *S12$ $S12 \rightarrow CS12$ $S12 \rightarrow *S13$ $S13 \rightarrow CS12$ $S13 \rightarrow /S14$ $S14 \rightarrow \epsilon$	S14	Al detectar exactamente la secuencia de cierre */.

Operador Flecha	$S0 \rightarrow -S15$ $S15 \rightarrow >S16$ $S16 \rightarrow \epsilon$	S16	Al recibir el símbolo > inmediatamente después de un guión.
Símbolos de un carácter	$S0 \rightarrow +S17$ $S0 \rightarrow =S18$ $S0 \rightarrow \{S19$ $S0 \rightarrow \}S20$ $S0 \rightarrow (S21$ $S0 \rightarrow)S22$ $S0 \rightarrow ,S23$ $S0 \rightarrow :S24$ $S17 \rightarrow \epsilon$ $S18 \rightarrow \epsilon$ $S19 \rightarrow \epsilon$ $S20 \rightarrow \epsilon$ $S21 \rightarrow \epsilon$ $S22 \rightarrow \epsilon$ $S23 \rightarrow \epsilon$ $S24 \rightarrow \epsilon$	S17 A S24	Aceptación inmediata al leer un símbolo válido de un solo carácter.

Nomenclatura utilizada: **L** (Letra), **D** (Dígito), **C** (Cualquier carácter), **epsilon** (Transición vacía / Aceptación del token).

Gramática Regular

Gramatica Regular

$G = N, T, P, S$

$N = \{ S_0, S_1, S_2, S_3, S_4, S_5, S_6, S_7, S_8, S_9, S_{10}, S_{11}, S_{12}, S_{13}, S_{14}, S_{15}, S_{16}, S_{17}, S_{18}, S_{19}, S_{20}, S_{21}, S_{22}, S_{23}, S_{24} \}$

$T = \{ L, D, C, @, ", /, *, -, >, +, =, \{, \}, (,), , , : , \backslash n, \epsilon \}$

$L = \text{letras}$

$D = \text{digitos}$

$C = \text{Qualquier Caracter}$

$P:$

$S_0 \rightarrow L S_1$	$S_1 \rightarrow L S_1$	$S_5 \rightarrow L S_6$	$S_{14} \rightarrow \epsilon$
$S_0 \rightarrow D S_2$	$S_1 \rightarrow D S_1$	$S_6 \rightarrow L S_6$	
$S_0 \rightarrow @ S_5$	$S_1 \rightarrow _ S_1$	$S_6 \rightarrow \epsilon$	$S_{15} \rightarrow > S_{16}$
$S_0 \rightarrow " S_7$	$S_1 \rightarrow \epsilon$		$S_{16} \rightarrow \epsilon$
$S_0 \rightarrow / S_9$		$S_7 \rightarrow C S_7$	
$S_0 \rightarrow - S_{15}$	$S_2 \rightarrow D S_2$	$S_7 \rightarrow " S_8$	$S_{17} \rightarrow \epsilon$
$S_0 \rightarrow = S_{17}$	$S_2 \rightarrow . S_3$	$S_8 = \epsilon$	$S_{18} \rightarrow \epsilon$
$S_0 \rightarrow + S_{18}$	$S_2 \rightarrow \epsilon$		$S_{19} \rightarrow \epsilon$
$S_0 \rightarrow \{ S_{19}$		$S_9 \rightarrow / S_{10}$	$S_{20} \rightarrow \epsilon$
$S_0 \rightarrow \} S_{20}$	$S_3 \rightarrow D S_4$	$S_9 \rightarrow * S_{12}$	$S_{21} \rightarrow \epsilon$
$S_0 \rightarrow (S_{21}$		$S_{10} \rightarrow (S_{10}$	$S_{22} \rightarrow \epsilon$
$S_0 \rightarrow) S_{22}$	$S_4 \rightarrow D S_4$	$S_{10} \rightarrow \backslash n S_{11}$	$S_{23} \rightarrow \epsilon$
$S_0 \rightarrow , S_{23}$	$S_4 \rightarrow \epsilon$	$S_{11} \rightarrow \epsilon$	$S_{24} \rightarrow \epsilon$
$S_0 \rightarrow : S_{24}$		$S_{12} \rightarrow C S_{12}$	
		$S_{12} \rightarrow * S_{13}$	
		$S_{13} \rightarrow C S_{12}$	
		$S_{13} \rightarrow / S_{14}$	

Recuperación de Errores

Cuando encontramos un error o un token no valido lo guardamos en una lista llamada `listaErrores` que nos va a servir para después mostrar los caracteres no validos al usuario y después de guardar los errores en esa lista seguimos avanzando para seguir leyendo el resto del archivo.

```
57         extraerComentariosDivision(),
58     } else {
59         listaErrores.add(new ErrorLexico(String.valueOf(c), descripcion: "Carácter no reconocido", fila, columna));
60         posicion++;
61         columna++;
62     }
63 }
64 }
65 }
```

```
--- ERRORES ENCONTRADOS ---

Error: '%' | Descripcion: Carácter no reconocido | Fila: 15 | Columna: 22
Error: '#' | Descripcion: Carácter no reconocido | Fila: 17 | Columna: 69
Error: '$' | Descripcion: Carácter no reconocido | Fila: 19 | Columna: 58
Error: '$' | Descripcion: Carácter no reconocido | Fila: 19 | Columna: 59
Error: '$' | Descripcion: Carácter no reconocido | Fila: 19 | Columna: 60
Error: 'Eres un redactor creativo' | Descripcion: Cadena sin cerrar | Fila: 23 | Columna: 14
Error: '?' | Descripcion: Carácter no reconocido | Fila: 28 | Columna: 46
Error: '&' | Descripcion: Carácter no reconocido | Fila: 34 | Columna: 24
```